



Proyecto - Control de Préstamos

José Daniel Mora Zúñiga

IC-2101 Programación Orientada a Objetos - Prof. Mauricio Avilés



Introducción

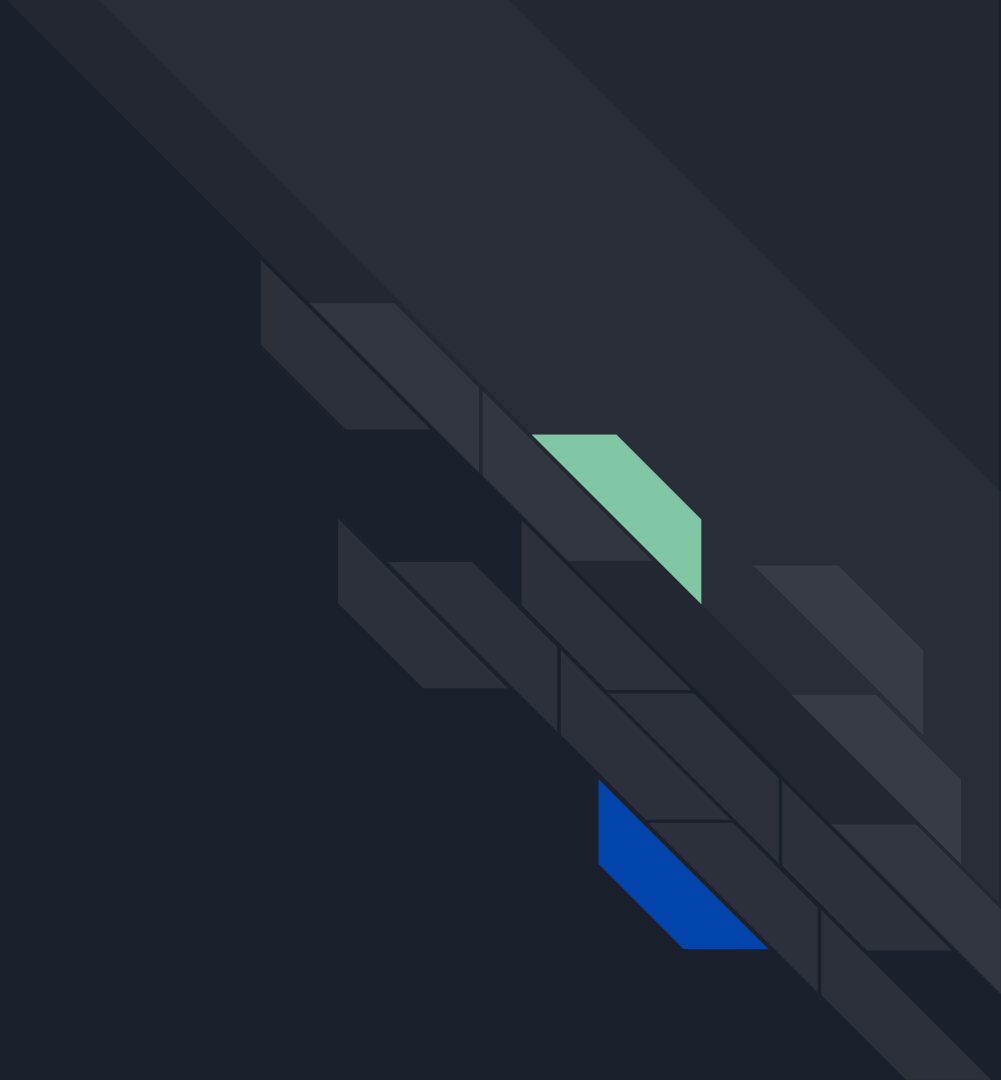
El proyecto consiste en una aplicación de escritorio en Java para gestionar préstamos de objetos. Se desarrolla para resolver el problema de desorganización y pérdida de objetos prestados.



El sistema permite:

- Administrar personas, ítems, tipos, categorías y préstamos, con alertas programables, reportes y persistencia.

Problemas:





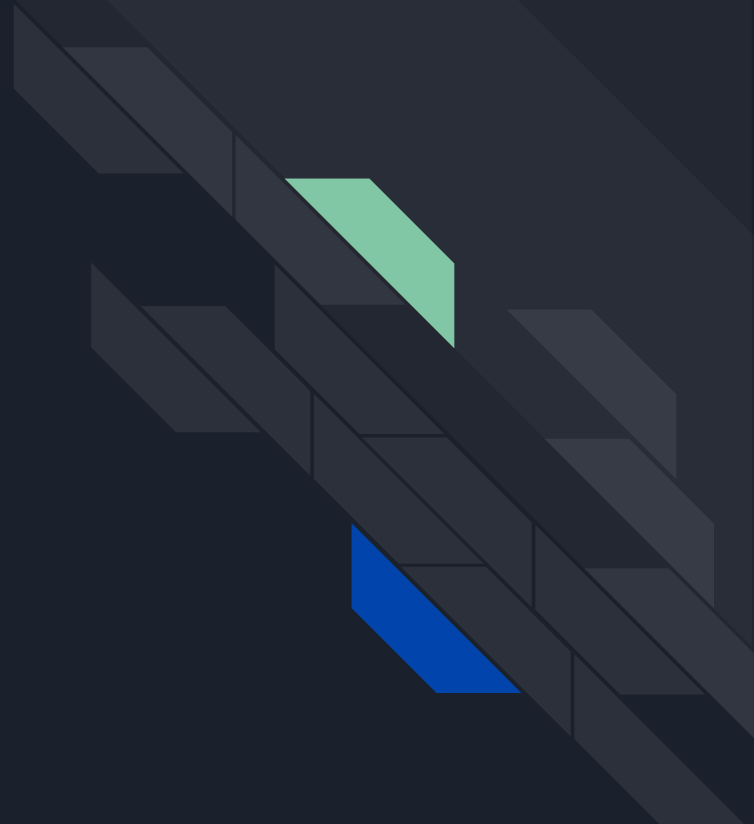
Principal

Hay que crear un sistema que registre quién tiene cada objeto, cuándo se prestó y cuándo debe devolverse. Esto genera extravíos, olvidos y falta de control.

Subproblemas

- Manipulación de personas (clave: teléfono).
- Manipulación de ítems (clave: código, asociación a tipo y categorías).
- Manipulación de tipos (con tipo genérico protegido).
- Manipulación de categorías (relación muchos a muchos con ítems).
- Gestión de préstamos (crear, agregar/quitar ítems, finalizar, retornar).
- Alertas (únicas o recurrentes al iniciar la app).
- Reportes (por usuario, ítem, categoría, tipo).
- Persistencia (guardado/carga en disco).
- Interfaz gráfica usable.

Metodología





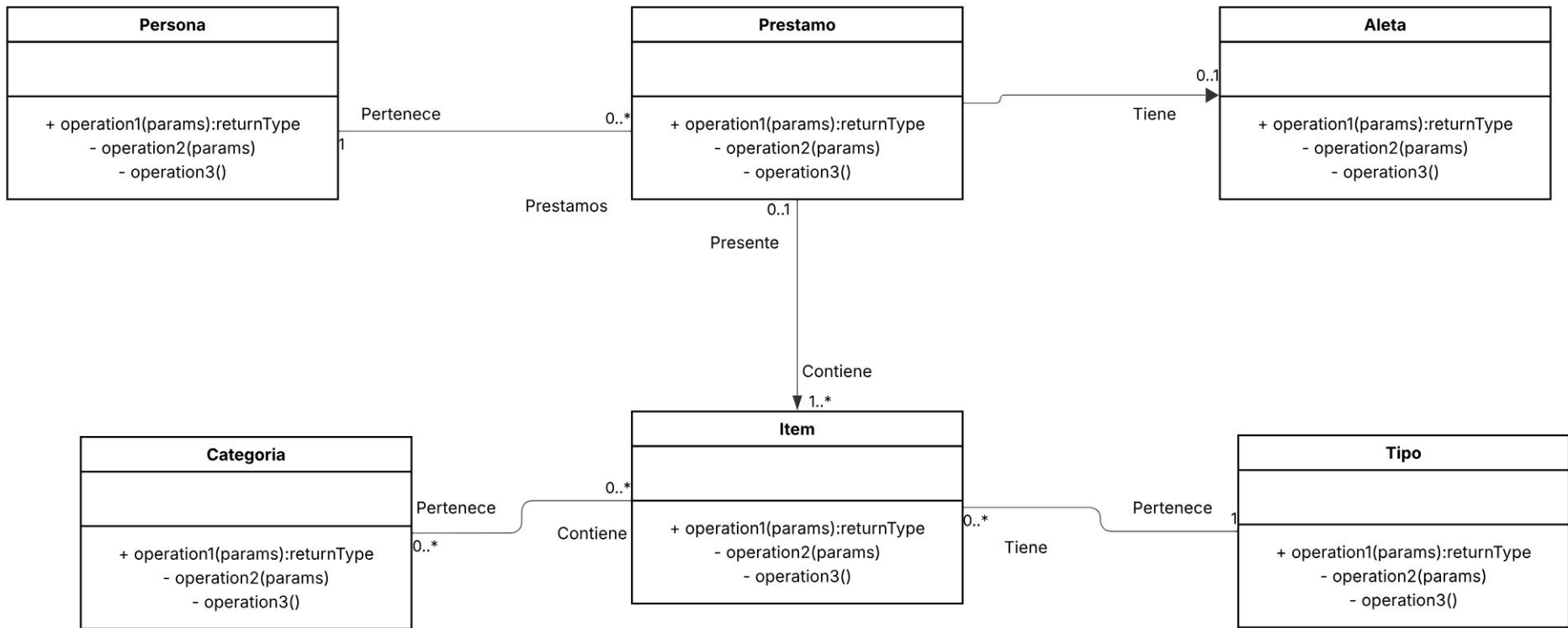
Clases principales

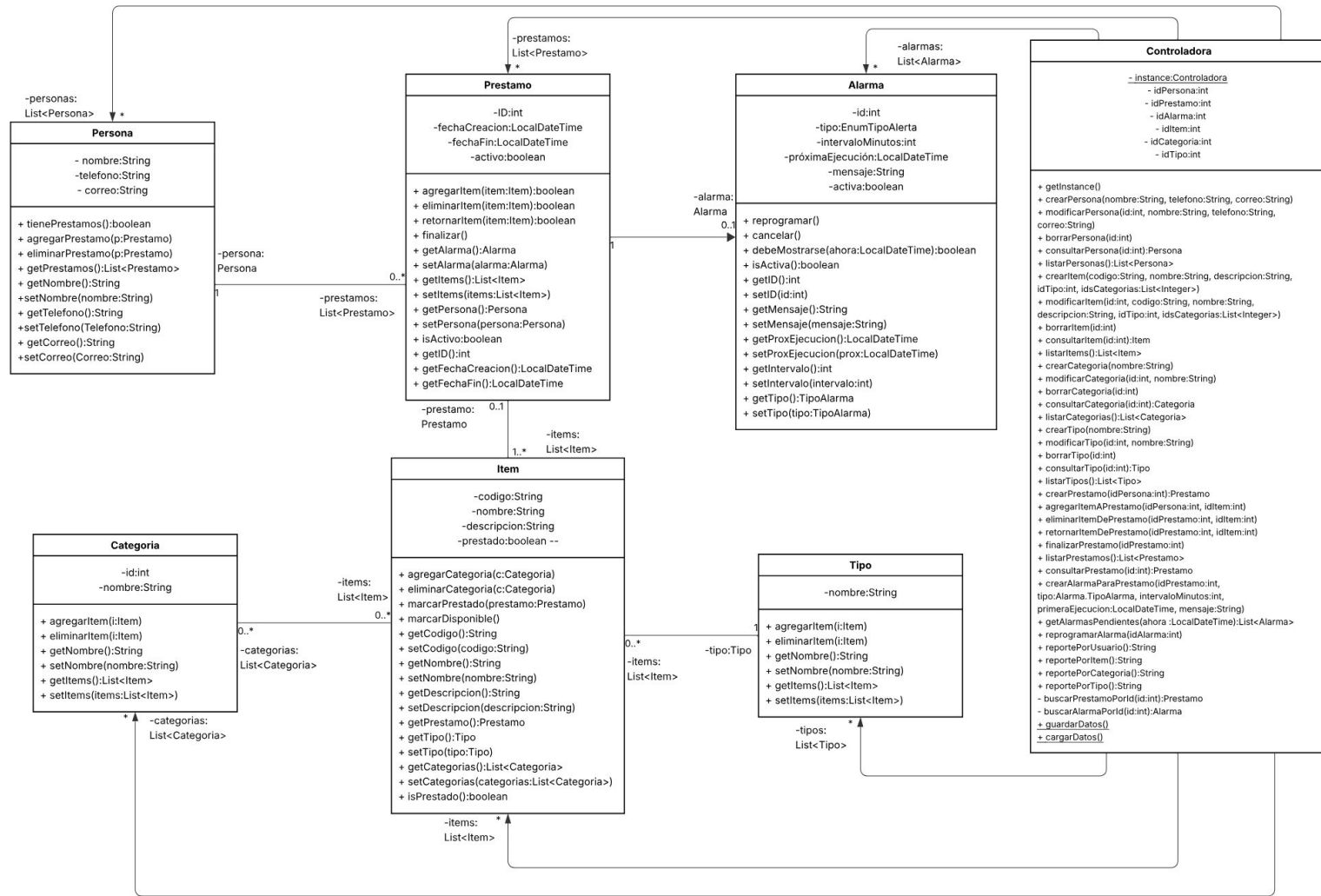
- Personas: `TreeMap<String, Persona>` (teléfono).
- Ítems: `TreeMap<String, Item>` (código), relación bidireccional con `Prestamo`.
- Tipos: `TreeMap<String, Tipo>`, protección para "Genérico".
- Categorías: `TreeMap<String, Categoria>`, relación muchos a muchos con `Item`.
- Préstamos: clase `Prestamo` con lista de `Item`, `Alarma` y métodos de gestión.
- Alertas: clase `Alarma`, `Timer` en interfaz para verificar cada minuto.
- Controladora: contiene los métodos para manipular todas las clases anteriores.
- Reportes: métodos en controladora que devuelven `String` con listados formateados.
- Interfaz: Java Swing con `JTabbedPane`, `JTable`, `JList`, `DefaultTableModel`.



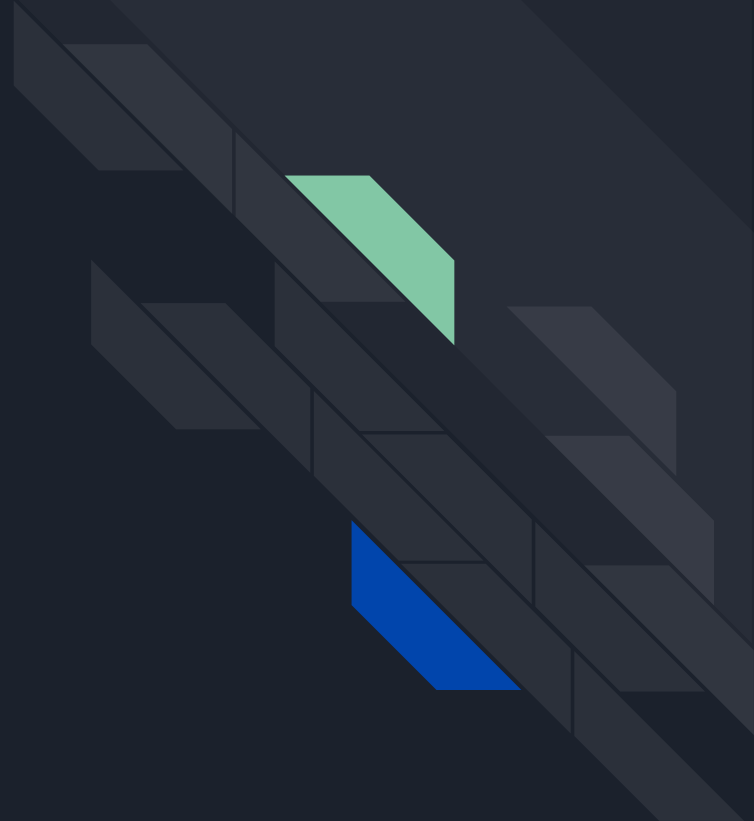
Persistencia

Persistencia: serialización Java (Serializable).





Análisis crítico





¿Qué se logró implementar?

- Administración completa de personas, ítems, tipos y categorías.
- Gestión de préstamos (crear, agregar/quitar, finalizar, retornar).
- Alertas únicas y recurrentes con verificación periódica.
- Reportes en texto (usuario, ítem, categoría, tipo).
- Persistencia por serialización.
- Interfaz con pestañas y subpestañas.



¿Qué faltó implementar?

- Validaciones de formato (correo, teléfono) se pueden mejorar.
- Exportación de reportes a PDF/CSV.
- Validaciones de entrada insuficientes.
- Limitaciones en la interfaz de usuario.
- No hay gestión de préstamos finalizados.
- Persistencia frágil



Conclusiones

La aplicación cubre todos los requisitos funcionales (personas, ítems, tipos, categorías, préstamos, alertas, reportes y persistencia).

La serialización es válida para proyectos pequeños, pero frágil ante cambios de clase.

TreeMap facilita el ordenamiento y búsqueda por clave natural.

El patrón Singleton centraliza la lógica y simplifica el acceso.

La interfaz basada en pestañas es organizada y usable.



Conclusiones

El modelo de datos permite extensiones (ej. nuevos atributos, tipos de alertas).

La falta de validaciones de entrada (correo, teléfono, códigos) puede provocar datos inconsistentes.

El manejo de excepciones es básico; los mensajes de error no siempre son claros para el usuario final.

No hay confirmación visual clara de las operaciones (ej. animaciones o mensajes de éxito persistentes).



Recomendaciones

Para asegurar que todo siga funcionando al añadir nuevas funciones, haga pruebas frecuentes de cada módulo por separado.

Si cambia la estructura de las clases, los datos guardados podrían no leerse.

Aproveche que TreeMap ordena los datos automáticamente para mostrar listados ordenados sin tener que programar ordenamientos adicionales.

Si la aplicación crece, divida la lógica en partes más independientes (como servicios) para que sea más fácil de mantener y probar.

Para mejorar la experiencia del usuario, agregue atajos de teclado y pruebe la interfaz con usuarios reales para ver qué flujos son más cómodos.



Recomendaciones

Piense con anticipación en cómo añadir nuevos tipos de objetos o alertas sin tener que reescribir todo el código; por ejemplo, usando interfaces o configuraciones externas.

Implemente validaciones en los campos (ej. que el correo tenga @, que el teléfono solo tenga números) y muestre mensajes de error específicos.

Mejore el manejo de excepciones con bloques try-catch más específicos y mensajes de error en español claros; considere un sistema de logs simple (archivo de texto) para registrar errores.

Muestre mensajes de éxito que sean temporales y no solo en ventanas emergentes, para una mejor experiencia.